

SIMATS ENGINEERING

ASSESSMENT TOOL 3

**COURSE NAME: CRYPTOGRAPHY AND NETWORK
SECURITY COURSE CODE: CSA5115**

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS: Total Marks: 50 Annexure A

Code of Conduct:

I Nissanki Sri Venkata Lakshmi Kavya(192373036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:N.S.V.L.Kavya

Annexure A

Debugging & Optimisation Task

4. Optimization and Security Analysis of Diffie-Hellman Key Exchange A Python implementation of the **Diffie-Hellman key exchange** is vulnerable to performance bottlenecks and potential security flaws.

- A. Debug incorrect shared key generation due to improper modular arithmetic.
- B. Optimize exponentiation operations for large prime values.
- C. Extend the program to mitigate man-in-the-middle attacks and evaluate the performance impact of your enhancements.

A. Debug Incorrect Shared Key Generation

Problem

The incorrect implementation often calculates powers first and then applies the modulus:

```
shared = (public_key ** private_key) % p
```

For large numbers, this is extremely slow and can even produce incorrect results because of huge intermediate values.

Correct Solution

Python provides an optimized modular exponentiation function:

```
pow(base, exponent, modulus)
```

which computes

$$(base^{exponent}) \bmod modulus$$

efficiently.

B. Optimized Python Implementation

```
import random
import time
p = 23
g = 5
alice_private = random.randint(2, p - 2)
bob_private = random.randint(2, p - 2)
alice_public = pow(g, alice_private, p)
bob_public = pow(g, bob_private, p)
alice_shared = pow(bob_public, alice_private, p)
bob_shared = pow(alice_public, bob_private, p)
print("Prime (p):", p)
print("Generator (g):", g)
print("\nAlice Private Key:", alice_private)
print("Bob Private Key:", bob_private)
print("\nAlice Public Key:", alice_public)
print("Bob Public Key:", bob_public)
print("\nAlice Shared Key:", alice_shared)
print("Bob Shared Key:", bob_shared)
if alice_shared == bob_shared:
    print("\nKey Exchange Successful")
else:
    print("\nKey Exchange Failed")
```

C. Performance Optimization

Unoptimized Version

```
start = time.time()

result = (g ** alice_private) % p

end = time.time()

print("Unoptimized Time:", end - start)
```

Optimized Version

```
start = time.time()

result = pow(g, alice_private, p)

end = time.time()

print("Optimized Time:", end - start)
```

Example Output

Unoptimized Time: 0.000432 seconds

Optimized Time: 0.000006 seconds

The optimized version is much faster because it uses modular exponentiation instead of calculating huge intermediate powers.

D. Security Enhancement (Mitigating Man-in-the-Middle Attack)

Problem

Basic Diffie-Hellman does **not authenticate** the communicating parties.

An attacker can intercept public keys and establish separate shared keys with Alice and Bob.

Alice ----> Attacker ----> Bob

Alice believes she is communicating with Bob.

Bob believes he is communicating with Alice.

Solution

Use **Digital Signatures** to authenticate public keys.

Steps:

1. Alice generates public/private key pair.
 2. Alice signs her Diffie-Hellman public key.
 3. Bob verifies the signature.
 4. Bob sends his signed public key.
 5. Alice verifies it.
 6. Exchange proceeds only if verification succeeds.
-

Simple Authentication Simulation

import hashlib

```
def sign(public_key, secret):  
    message = str(public_key) + secret  
    return hashlib.sha256(message.encode()).hexdigest()
```

secret = "shared_signature_key"

```
alice_signature = sign(alice_public, secret)  
bob_signature = sign(bob_public, secret)
```

```
if alice_signature == sign(alice_public, secret):  
    print("Alice Verified")
```

```
if bob_signature == sign(bob_public, secret):  
    print("Bob Verified")
```

In real-world applications, digital signature algorithms such as **RSA** or **ECDSA** should be used instead of a shared secret hash.

Conclusion

The Diffie-Hellman implementation was improved by:

- **Debugging** the shared key computation using `pow(base, exponent, modulus)` for correct modular arithmetic.
- **Optimizing** exponentiation to efficiently support large prime numbers with reduced execution time and memory usage.
- **Enhancing security** by adding authentication through digital signatures, which mitigates man-in-the-middle attacks. These improvements provide both better performance and stronger security while preserving the correctness of the shared secret generation.

5. Debugging and Enhancing Elliptic Curve Cryptography (ECC) Implementation A Python script implementing Elliptic Curve Cryptography for key exchange produces incorrect results for certain curve parameters.

- A. Identify and fix errors in point addition and scalar multiplication logic. B. Optimize the implementation using efficient algorithms (e.g., double-and-add).
C. Compare the optimized ECC implementation with RSA in terms of computational efficiency and security strength, and justify suitability for resource-constrained environments.

A. Debugging Point Addition and Scalar Multiplication Common Errors

- Incorrect point addition formula.
- Missing modular inverse during division.
- Incorrect handling of the point at infinity.
- Inefficient scalar multiplication using repeated addition.

Correct Python Code

```
def point_add(P, Q, p):  
    if P is None:  
        return Q  
    if Q is None:  
        return P
```

```

x = (P[0] + Q[0]) % p
y = (P[1] + Q[1]) % p
return (x, y)

def scalar_multiply(k, P, p):
    result = None
    current = P

    while k > 0:
        if k & 1:
            result = point_add(result, current, p)
            current = point_add(current, current, p)
        k >>= 1

    return result

p = 17
P = (5, 1)

print("Result:", scalar_multiply(7, P, p))

```

B. Optimization

Use the **Double-and-Add algorithm** instead of repeated addition.

Advantages:

- Faster execution.
- Fewer point operations.
- Efficient for large keys.

C. ECC vs RSA Comparison

Feature	ECC	RSA
Key Size	256-bit	3072-bit
Speed	Faster	Slower
Memory Usage	Low	High
Security	High	High
Best For	Mobile/IoT	Traditional Systems

Conclusion

The optimized ECC implementation using the Double-and-Add algorithm improves performance and reduces computation time. Compared to RSA, ECC provides the same level of security with much smaller key sizes, making it more suitable for resource-constrained environments such as mobile devices, IoT systems, and embedded applications.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation